

11주차 과제

CLIP

1. 어떤 문제를 해결하려 했는가? (Abstract)

최신 컴퓨터 비전 시스템은 미리 정해진 고정된 객체 카테고리 집합을 예측하도록 학습된다. 이러한 제한적 지도 방식은 다른 시각적 개념을 지정하려면 추가 레이블 데이터가 필요하기 때문에 범용성과 활용성에 한계가 있다.

인터넷에서 수집한 4억 개의 (이미지, 텍스트) 쌍 데이터셋에서 어느 캡션이 어느 이미지와 쌍을 이루는지 예측하는 단순한 사전학습 태스크가 최고 수준의 이미지 표현을 처음부터 학습하는 효율적이고 확장 가능한 방법임을 보인다. 사전학습 후 자연어를 통해 학습된 시각적 개념을 참조하거나 새로운 개념을 기술함으로써 다운스트림 태스크로의 제로샷 전이가 가능해진다.

핵심 문제 정리:

기존 CV 패러다임의 한계	CLIP의 해결 방향
고정된 클래스 집합(e.g., ImageNet 1000개)	자연어로 무한히 열린 개념 표현
새 태스크마다 레이블 데이터 필요	텍스트 프롬프트만으로 제로샷 전이
특정 도메인에 과적합	인터넷 스케일 다양성으로 범용화
정적 분류기 (static softmax)	동적 텍스트 기반 분류기 생성

2. 연구 동기와 문제점 (Introduction)

NLP에서는 원시 텍스트에서 직접 학습하는 사전학습 방법이 지난 몇 년간 혁명을 일으켰다. 태스크 무관 목적 함수(autoregressive, masked language modeling)는 컴퓨팅, 모델 용량, 데이터 규모에 걸쳐 수십 배 이상으로 확장되며 꾸준히 성능이 향상되었다. GPT-3 같은 플래그십 시스템은 이제 데이터셋별 학습 데이터가 거의 없어도 맞춤 모델과 경쟁 가능하다.

이러한 결과는 웹 규모의 텍스트 컬렉션에서 현대 사전학습 방법이 접근 가능한 지도 신호의 총합이 고품질 군중 레이블 NLP 데이터셋을 능가한다는 점을 시사한다. 그러나 컴퓨터 비전 분야에서는 여전히 ImageNet 같은 군중 레이블 데이터셋으로 사전학습하는 것이 표준 관행이다.

문제의 근본: 스케일과 방법론의 불일치

약한 지도 모델과 자연어에서 직접 이미지 표현을 학습하는 최근 탐구 간의 결정적 차이는 스케일이다. Mahajan et al. (2018)과 Kolesnikov et al. (2019)은 수백만~수십억 이미지로 가속기 수년을 학습한 반면, VirTex, ICMLM, ConVIRT는 10~20만 이미지로 가속기 며칠만 학습했다.

3. 관련 연구 동향 (Related Works)

연구	핵심 기여	한계
Mori et al. (1999)	이미지와 텍스트 명사/형용사 예측	초기 개념, 성능 미흡
Joulin et al. (2016)	CNN으로 이미지 캡션 단어 예측	BoW 표현, 의미 손실
Visual N-Grams (Li et al., 2017)	제로샷 이미지 분류 첫 시도	ImageNet 11.5%에 불과
VirTex / ICMLM / ConVIRT (2020)	Transformer 기반 이미지-텍스트 표현	소규모 데이터, 확장성 부족
ALIGN (Jia et al., 2021)	18억 이미지-텍스트 쌍 노이즈 학습	동시기 병렬 연구
SimCLR / MoCo	이미지 자기지도 대조 학습	언어 연결 부재

CLIP의 포지셔닝: 위 연구들의 아이디어를 계승하되, 스케일과 대조 목적 함수의 결합으로 모든 선행 연구를 압도적으로 능가

4. 연구 접근법과 모델 구조 (Method)

4-1. 전체 파이프라인 (3단계)

표준 이미지 모델이 이미지 특징 추출기와 선형 분류기를 공동으로 학습해 레이블을 예측하는 반면, CLIP은 이미지 인코더와 텍스트 인코더를 공동 학습하여 배치 내 (이미지, 텍스트) 훈련 예시들의 올바른 쌍을 예측한다. 테스트 시에는 학습된 텍스트 인코더가 대상 데이터셋의 클래스 이름이나 설명을 임베딩하여 제로샷 선형 분류기를 합성한다.

4-2. 데이터셋: WIT (WebImageText)

기존 데이터셋들은 현대 기준으로 소규모이다(MS-COCO, Visual Genome 각 약 10만 장). 이를 해결하기 위해 인터넷의 다양한 공개 소스에서 수집한 **4억 개의 (이미지, 텍스트) 쌍** 데이터셋을 구축했다. 가능한 한 넓은 시각적 개념을 포괄하기 위해 50만 개의 쿼리에 해당하는 텍스트를 포함하는 쌍을 검색하고, 클래스 균형을 위해 쿼리당 최대 20,000 쌍을 포함했다.

4-3. 핵심: 대조 학습 목적 함수 선택 과정

초기 접근법은 VirTex처럼 이미지 CNN과 텍스트 트랜스포머를 공동 학습하여 이미지의 캡션을 예측하는 것이었다. 그러나 이 방법을 효율적으로 확장하는 데 어려움이 있었다. 63M 파라미터 트랜스포머 언어 모델이 이미 ResNet-50 이미지 인코더보다 2배의 컴퓨팅을 사용하면서도 BoW 인코딩을 예측하는 훨씬 단순한 베이스라인보다 3배 느리게 ImageNet 클래스를 인식했다.

두 접근법의 공통점은 각 이미지와 함께하는 텍스트의 정확한 단어를 예측하려 한다는 것이다. 이는 이미지와 공존하는 설명, 댓글, 관련 텍스트의 다양성 때문에 어려운 태스크다. 대조 목적 함수로 전환했을 때 제로샷 전이 효율이 추가로 4배 향상되는 것을 관찰했다.

4-4. 이미지 인코더 아키텍처

이미지 인코더로 두 가지 아키텍처를 고려했다. 첫째, ResNet-50을 기반으로 ResNet-D 개선점과 Antialiased rect-2 블러 풀링을 적용하고, 전역 평균 풀링을 어텐션 풀링으로 교체했다. 둘째, 최근 도입된 Vision Transformer(ViT)를 실험했다.

4-5. 텍스트 인코더 아키텍처

텍스트 인코더는 63M 파라미터, 12레이어, 512차원, 8개 어텐션 헤드의 Transformer이다. 텍스트는 49,152 어휘 크기의 소문자 BPE 표현으로 처리되며, 최대 시퀀스 길이는 76으로 제한된다. [EOS] 토큰 위치의 최상위 레이어 활성화가 텍스트 특징 표현으로 사용된다.

4-6. 스케일링 전략

ResNet 이미지 인코더의 경우 폭, 깊이, 해상도 전체에 걸쳐 컴퓨팅을 균등하게 할당하는 방식으로 확장했다. 텍스트 인코더의 경우 깊이는 전혀 확장하지 않고 폭만 ResNet 폭 증가에 비례하여 확장했는데, CLIP 성능이 텍스트 인코더 용량에 덜 민감하다는 것을 발견했기 때문이다.

5. 실험 구성과 결과 (Experiments)

5-1. 훈련 설정

5개의 ResNet과 3개의 Vision Transformer를 훈련했다. 모든 모델은 32 에포크 동안 학습하며, Adam 옵티마이저와 분리된 가중치 감쇠 규제, 코사인 학습률 스케줄을 사용했다. 미니배치 크기는 매우 큰 32,768을 사용했다. 가장 큰 ResNet 모델(RN50×64)은 592개의 V100 GPU로 18일, 가장 큰 Vision Transformer는 256개의 V100으로 12일이 걸렸다.

5-2. 제로샷 전이 성능

최고 CLIP 모델은 ImageNet에서의 정확도를 기존 최선의 제로샷 방법인 11.5%에서 76.2%로 향상시켰으며, 1.28M 크라우드 레이블 학습 예시를 전혀 사용하지 않고도 원본

ResNet-50의 성능에 맞먹는다.

방법	aYahoo	ImageNet	SUN
Visual N-Grams	72.4%	11.5%	23.0%
CLIP	98.4%	76.2%	58.5%

5-3. 프롬프트 엔지니어링 효과

프롬프트 엔지니어링과 앙상블은 36개 데이터셋에서 평균 약 5포인트 제로샷 분류 성능을 향상시킨다. 이 향상은 베이스라인 제로샷 방법에 4배 더 많은 컴퓨팅을 사용했을 때의 이득과 유사하지만, 많은 예측에 분산될 때 "무료"로 얻을 수 있다.

구체적 프롬프트 전략:

기본 프롬프트로 "A photo of a {label}." 이 좋은 기본값임을 발견했다. ImageNet에서 이 프롬프트만으로 정확도가 1.3% 향상된다. 세밀 분류 데이터셋에서는 카테고리 맥락을 명시하면 도움이 되며(예: "A photo of a {label}, a type of pet."), 위성 이미지의 경우 이미지 형식을 명시하는 것이 효과적이다. ImageNet에서 80가지 맥락 프롬프트를 앙상블하면 추가로 3.5%가 향상된다.

5-4. 선형 프로브 분석

CLIP은 최고의 공개 ImageNet 모델보다 높은 성능을 보이면서도 계산 효율이 더 높다.

5-5. 분포 이동 견고성

제로샷 CLIP 모델이 동등한 정확도의 지도학습 ImageNet 모델보다 훨씬 견고하다는 것을 추가로 발견했으며, 이는 태스크 무관 모델의 제로샷 평가가 모델 능력을 훨씬 잘 대표함을 시사한다.

6. 발견사항과 한계점 (Discussion)

주요 발견

- **텍스트가 이미지보다 더 풍부한 지도 신호:** 자연어는 고정된 카테고리 집합을 초월하여 무한한 시각적 개념을 표현
- **스케일 법칙 적용:** 컴퓨팅 증가에 따라 전이 성능이 예측 가능하게 향상
- **컴포지셔널리티:** DALL·E 등 다른 시스템의 핵심 구성 요소로 활용 가능

한계점

CLIP이 제로샷으로 어려움을 겪는 태스크들도 있다. OCR, 비디오 내 행동 인식, 지리적 위치 파악 등에서 인상적인 성능을 보이지만, 세밀 분류(차량 모델, 꽃 종류 등)나 추상 개념(물체 수 세기), 분포 외 태스크(MNIST 손글씨)에서는 어려움이 있다.

구체적 한계 정리:

한계 유형	내용
세밀 분류	차종, 꽃 종류 등 세부 구별 어려움
숫자 세기	"3개의 사과" 등 정량적 이해 취약
MNIST	훈련 분포 외 단순 태스크에서도 실패
사회적 편향	웹 데이터의 편향을 그대로 흡수
효율성	4억 쌍 데이터 + 수백 GPU 필요
공정성 문제	인종, 성별 등 민감 속성에 대한 편향 가능성

7. 결론 및 주요 요약 (Conclusion)

CLIP의 핵심 기여

1. **자연어 지도의 실용적 확장**: 4억 쌍 데이터 + 대조 학습으로 NLP 혁명을 CV로 이식
2. **제로샷 패러다임 정립**: 새 태스크 = 텍스트 프롬프트 설계 문제로 재정의
3. **견고성 향상**: 지도학습 모델 대비 분포 이동에 훨씬 강함
4. **범용 비전 백본**: DALL·E, SAM, Stable Diffusion 등 수많은 후속 연구의 핵심 구성 요소
5. **오픈소스**: 코드 및 사전학습 가중치 공개

8. 기존 연구 대비 차별점

비교 항목	기존 방법 (ResNet/ViT)	기존 약지도 (Mahajan 등)	CLIP
학습 신호	고정 레이블 (1000개)	약한 레이블 (18291개)	자연어 (무한)
제로샷 능력	없음	제한적	강력한 제로샷
동적 분류기	불가	불가	텍스트로 즉시 생성
데이터 수집	수작업 레이블링	해시태그/약한레이블	인터넷 무감독 수집
전이 견고성	낮음	중간	매우 높음
목적 함수	교차 엔트로피 분류	예측	대칭 대조 손실

$$\mathcal{L} = \frac{1}{2} (\mathcal{L}_{image} + \mathcal{L}_{text})$$

$$\mathcal{L}_{image} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s_{ii}}}{\sum_{j=1}^N e^{s_{ij}}}$$

$$\mathcal{L}_{text} = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s_{ii}}}{\sum_{j=1}^N e^{s_{ji}}}$$

- $\mathcal{L}_{image}$: 각 이미지에 대해 올바른 텍스트를 찾는 손실 (행 방향 softmax)
- $\mathcal{L}_{text}$: 각 텍스트에 대해 올바른 이미지를 찾는 손실 (열 방향 softmax)

이는 **InfoNCE 손실**과 동일하며, **multi-class N-pair loss**의 대칭 버전

온도 파라미터 τ 의 역할:

- τ 가 작을수록 → 유사도 분포가 뽀족 → 더 discriminative
- $\tau = 0.07$ 로 초기화, 학습 중 자동 최적화
- `np.exp( $\tau$ )` 로 클리핑하여 학습 불안정 방지

10-2. 논문 원본 의사코드

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) # [n, d_i]
T_f = text_encoder(T)  # [n, d_t]
```

```

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0) # 행
          방향
loss_t = cross_entropy_loss(logits, labels, axis=1) # 열
          방향
loss = (loss_i + loss_t) / 2

```

10-3. PyTorch 구현

```

import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision.models import resnet50
from transformers import BertModel, BertTokenizer

# — 1. CLIP 모델 정의 —————

class CLIPModel(nn.Module):
    """
    CLIP: 이미지-텍스트 대조 학습 모델
    - 이미지 인코더: ResNet-50 (또는 ViT)
    - 텍스트 인코더: Transformer
    - 공유 임베딩 공간에서 코사인 유사도 최대화/최소화
    """

    def __init__(self, embed_dim=512):
        super().__init__()

```



```

# — 이미지 인코더 —————
# ResNet-50을 기반으로, 마지막 FC 레이어를 선형 프로젝션
# 으로 교체
self.image_encoder = resnet50(pretrained=False)
d_image = self.image_encoder.fc.in_features # 2048
self.image_encoder.fc = nn.Identity()      # FC
# 제거

# 이미지 임베딩 → 공유 임베딩 공간으로 선형 프로젝션
# 논문: "only a linear projection to map from each
# encoder's
#         representation to the multi-modal embeddin
# g space"
self.image_projection = nn.Linear(d_image, embed_dim, bias=False)

# — 텍스트 인코더 —————
# 63M 파라미터 Transformer (12레이어, 512차원, 8헤드)
# 실제 CLIP은 GPT-2 스타일이지만, 여기서는 BERT로 단순화
self.text_encoder = BertModel.from_pretrained('bert
-base-uncased')
d_text = 768 # BERT-base 차원

# 텍스트 임베딩 → 공유 임베딩 공간으로 선형 프로젝션
self.text_projection = nn.Linear(d_text, embed_dim,
bias=False)

# — 학습 가능한 온도 파라미터  $\tau$  —————
# 논문: "directly optimized during training as a lo
g-parameterized
#         multiplicative scalar"
#  $\log\_scale = \log(1/\tau)$ , 초기값  $\log(1/0.07) \approx 2.659$ 
self.logit_scale = nn.Parameter(torch.ones([]) * 2.
659)

```

```

        # 클리핑: logits이 100을 초과하지 않도록 (학습 불안정 방
지)
        self.logit_scale_max = 100.0

    def encode_image(self, images):
        """
        이미지를 임베딩으로 변환
        images: [N, C, H, W]
        return: [N, embed_dim] (L2 정규화됨)
        """
        image_features = self.image_encoder(images)
# [N, 2048]
        image_features = self.image_projection(image_featur
es) # [N, embed_dim]
        image_features = F.normalize(image_features, dim=-
1) # L2 정규화
        return image_features

    def encode_text(self, input_ids, attention_mask):
        """
        텍스트를 임베딩으로 변환
        input_ids: [N, L] (BPE 토큰 ID)
        return: [N, embed_dim] (L2 정규화됨)
        """
        outputs = self.text_encoder(
            input_ids=input_ids,
            attention_mask=attention_mask
        )
        # [CLS] 토큰 임베딩 추출 (CLIP 원본은 [EOS] 위치 사용)
        text_features = outputs.last_hidden_state[:, 0, :]
# [N, 768]
        text_features = self.text_projection(text_features)
# [N, embed_dim]
        text_features = F.normalize(text_features, dim=-1)
# L2 정규화
        return text_features

```

```

def forward(self, images, input_ids, attention_mask):
    """
    순전파: 이미지-텍스트 쌍의 대칭 대조 손실 계산
    """
    N = images.shape[0]

    # — Step 1: 임베딩 추출 —————
    image_features = self.encode_image(images)
    text_features = self.encode_text(input_ids, attention_mask)

    # — Step 2: 스케일된 코사인 유사도 행렬 계산 —————
    # logit_scale = exp(log_scale)  $\approx 1/\tau$ 
    logit_scale = self.logit_scale.exp().clamp(max=self.logit_scale_max)

    #  $[N, D] \times [D, N] = [N, N]$  유사도 행렬
    logits = image_features @ text_features.T * logit_scale

    # — Step 3: 대칭 교차 엔트로피 손실 —————
    labels = torch.arange(N, device=images.device)

    # 이미지 방향: 각 이미지 i에 대해 올바른 텍스트 j=i 찾기
    (행) loss_image = F.cross_entropy(logits, labels)

    # 텍스트 방향: 각 텍스트 j에 대해 올바른 이미지 i=j 찾기
    (열) loss_text = F.cross_entropy(logits.T, labels)

    # 대칭 손실: 두 방향 평균
    loss = (loss_image + loss_text) / 2

    return loss, logits

```

— 2. 제로샷 분류 파이프라인

```
class CLIPZeroShotClassifier:
    """
    CLIP을 이용한 제로샷 분류기
    - 클래스 이름을 텍스트 프롬프트로 변환
    - 이미지와의 코사인 유사도로 분류
    """

    def __init__(self, clip_model, tokenizer, device='cuda'):
        self.model = clip_model.eval()
        self.tokenizer = tokenizer
        self.device = device

    def create_zero_shot_classifier(self, class_names, templates=None):
        """
        클래스 이름들로부터 텍스트 임베딩 기반 분류기 생성
        """
        if templates is None:
            # 기본 프롬프트: 논문 권장 템플릿
            templates = ["A photo of a {}."]

        all_text_embeddings = []

        with torch.no_grad():
            for class_name in class_names:
                class_embeddings = []

                for template in templates:
                    text = template.format(class_name)

                    tokens = self.tokenizer(
                        text,
```

```

        max_length=76,          # CLIP 최대 시퀀
스 길이

        padding='max_length',
        truncation=True,
        return_tensors='pt'
    ).to(self.device)

    text_emb = self.model.encode_text(
        tokens['input_ids'],
        tokens['attention_mask']
    ) # [1, embed_dim]

    class_embeddings.append(text_emb)

    # 임베딩 공간에서 평균 앙상블
    # 논문: "We construct the ensemble over the
embedding space
    #         instead of probability space"
    class_emb = torch.stack(class_embeddings).m
ean(0)

    class_emb = F.normalize(class_emb, dim=-1)
    all_text_embeddings.append(class_emb)

    # [num_classes, embed_dim] 텍스트 임베딩 행렬
    return torch.cat(all_text_embeddings, dim=0)

def predict(self, images, text_embeddings):
    """
    이미지를 분류
    images:          [N, C, H, W]
    text_embeddings:  [num_classes, embed_dim]
    return:          [N, num_classes] 예측 확률
    """
    with torch.no_grad():
        image_features = self.model.encode_image(image
s)
        # [N, D]
        logit_scale = self.model.logit_scale.exp()
        logits = image_features @ text_embeddings.T * l

```

```

ogit_scale # [N, C]
        probs = F.softmax(logits, dim=-1)
        return probs

# — 3. 실제 사용 예시 (openai/clip 라이브러리 활용) —————
_____

def clip_zero_shot_example():
    """
    CLIP 제로샷 분류 실제 사용 예시
    pip install git+https://github.com/openai/CLIP.git
    """

    import clip
    from PIL import Image
    import requests

    device = "cuda" if torch.cuda.is_available() else "cpu"

    # — 모델 로드 —————
    _____

    # ViT-L/14@336px이 논문에서 최고 성능
    model, preprocess = clip.load("ViT-L/14@336px", device=
device)

    # — 이미지 준비 —————
    _____

    url = "https://example.com/dog.jpg"
    image = Image.open(requests.get(url, stream=True).raw)
    image_tensor = preprocess(image).unsqueeze(0).to(device)

    # — 프롬프트 템플릿 앙상블 (논문 권장 방식) —————
    _____

    class_names = ["cat", "dog", "bird", "fish", "horse"]
    templates = [
        "a photo of a {}.",
        "a good photo of a {}.",

```

```

        "a photo of a {} in the wild.",
        "an image of a {}.",
    ]

    text_inputs = torch.cat([
        clip.tokenize([t.format(c) for t in templates])
        for c in class_names
    ]).to(device)

    with torch.no_grad():
        # 이미지 인코딩
        image_features = model.encode_image(image_tensor)
        image_features = F.normalize(image_features, dim=-
1)

        # 텍스트 인코딩 (각 클래스 × 각 템플릿)
        text_features = model.encode_text(text_inputs)
        text_features = F.normalize(text_features, dim=-1)

        # 클래스별 평균 임베딩 (양상블)
        text_features = text_features.view(
            len(class_names), len(templates), -1
        ).mean(dim=1) # [num_c
lasses, embed_dim]
        text_features = F.normalize(text_features, dim=-1)

        # 유사도 계산 및 분류
        similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)

        # 결과 출력
        for class_name, prob in zip(class_names, similarity
[0]):
            print(f"{class_name}: {prob.item():.3f}")

    predicted = class_names[similarity.argmax().item()]
    print(f"\n예측 클래스: {predicted}")

```

10-4. 제로샷 분류의 수학적 해석

제로샷 분류 = 하이퍼네트워크가 생성하는 동적 선형 분류기

입력: 이미지 I 클래스 텍스트 $\{T_1, T_2, \dots, T_k\}$

1. 이미지 임베딩: $f(I) \in \mathbb{R}^d$ (L2 정규화)

2. 텍스트 임베딩 (분류기 가중치):

$w_c = g(T_c) \in \mathbb{R}^d$ (L2 정규화)

→ 텍스트 인코더가 "하이퍼네트워크"로 동작

3. 예측:

$p(c | I) = \text{softmax}([f(I)^T \cdot w_1, f(I)^T \cdot w_2, \dots, f(I)^T \cdot w_k] / \tau)$

- L2 정규화 입력 ($f(I)$)
- L2 정규화 가중치 (w_c)
- 편향(bias) 없음
- 온도 스케일링(temperature scaling)

CLIP의 제로샷 분류기는 "바이어스 없는 다항 로지스틱 회귀 분류기" → 가중치를 자연어로 동적 생성한다는 것이 차별점

SAM

1. 어떤 문제를 해결하려 했는가? (Abstract)

이 논문은 이미지 세그멘테이션을 위한 새로운 태스크, 모델, 데이터셋을 통해 **비전 분야의 파운데이션 모델**을 구축하려는 시도이다.

핵심 문제:

문제	내용
데이터 부족	세그멘테이션 마스크는 인터넷에 자연적으로 풍부하지 않음
일반화 한계	기존 세그멘테이션 모델은 특정 태스크·도메인에 종속적
제로샷 불가	NLP와 달리 CV에는 프롬프트 기반 범용 세그멘테이션 모델이 없었음

2. 연구 동기와 문제점 (Introduction)

NLP에서 대규모 웹 데이터로 사전학습된 언어 모델이 강력한 제로샷/퓨샷 일반화를 보이는 것에 영감을 받아, 세그멘테이션을 위한 동일한 수준의 파운데이션 모델을 구축하고자 했다.

1. **Task**: 어떤 태스크가 제로샷 일반화를 가능하게 하는가?
2. **Model**: 그에 맞는 모델 아키텍처는 무엇인가?
3. **Data**: 이 태스크와 모델을 위한 데이터는 어떻게 확보하는가?

세그멘테이션 마스크는 인터넷에서 자연적으로 수집할 수 없어, 모델을 데이터 수집 루프에 활용하는 데이터 엔진(Data Engine)을 직접 구축하는 방식을 선택했다.

3. 관련 연구 동향 (Related Works)

논문에서 명시적으로 언급된 관련 연구들을 분야별로 분류하면 다음과 같다:

분야	대표 연구	특징
비전-언어 파운데이션	CLIP, ALIGN	텍스트-이미지 대조학습
이미지 생성	DALL-E	CLIP을 구성 요소로 활용
인터랙티브 세그멘테이션	RITM, FocalClick, SimpleClick	사용자 점 입력 기반 마스크 예측
비전 트랜스포머	ViT, MAE	이미지 인코더 백본
인스턴스 세그멘테이션	Mask R-CNN, ViTDet	완전 지도학습 기반
에지 검출	HED, EDTER	특정 태스크 전용 모델
대규모 언어 모델	GPT-3, PaLM	프롬프팅 패러다임 제공

SAM의 포지셔닝: 기존 인터랙티브 세그멘테이션은 충분한 사용자 입력 후 하나의 마스크를 예측하는 것이 목표이지만, SAM은 **어떤 프롬프트에 대해서도 즉시 유효한 마스크를 예측**하는 범용 모델을 지향한다.

4. 연구 접근법과 모델 구조 (Method)

4-1. 태스크 정의: Promptable Segmentation

프롬프트는 세그멘테이션 대상을 지정하는 **점(foreground/background points)**, **박스(box)**, **마스크(mask)**, **자유형 텍스트(text)** 등 어떤 형태도 가능하며, 모델은 이에 대해 항상 유효한 마스크를 반환해야 한다. 프롬프트가 모호한 경우(예: 셔츠를 가리키는 점이 셔츠인지 사람인지 불분명할 때)에도 적어도 하나의 합리적인 마스크를 출력해야 한다.

4-2. SAM 모델 아키텍처

SAM은 세 가지 핵심 구성 요소로 이루어진다: **이미지 인코더(Image Encoder)**, **프롬프트 인코더(Prompt Encoder)**, **경량 마스크 디코더(Lightweight Mask Decoder)**.

각 구성 요소 세부 사항:

① Image Encoder

MAE 사전학습된 ViT-H/16을 사용하며, 1024×1024 입력을 받아 64×64 크기의 256채널 이미지 임베딩을 출력한다. 이미지당 한 번만 실행되어 비용이 분산(amortized)된다.

② Prompt Encoder

점과 박스는 위치 인코딩(positional encoding)에 학습된 임베딩을 더해 256차원 벡터로 표현하고, 텍스트는 CLIP의 텍스트 인코더를 사용한다. 밀집 프롬프트(마스크)는 합성곱 레이어로 처리 후 이미지 임베딩과 element-wise 덧셈한다.

③ Mask Decoder

디코더의 각 레이어는 4단계로 동작한다: (1) 토큰 간 self-attention, (2) 토큰 → 이미지 임베딩 cross-attention, (3) MLP로 각 토큰 업데이트, (4) 이미지 임베딩 → 토큰 cross-attention.

전체 모델에서 프롬프트 인코더와 마스크 디코더는 웹 브라우저 CPU 환경에서도 약 **50ms** 안에 실행 가능하도록 설계되었다.

4-3. 모호성 해결(Ambiguity Resolution)

단일 프롬프트가 여러 유효한 마스크에 대응될 수 있는 문제를 해결하기 위해, 모델은 기본적으로 **3개의 마스크를 동시에 예측**한다(전체, 부분, 세부 부분의 계층적 구조). 학습 시에는 예측된 마스크 중 최소 손실만 역전파한다.

4-4. Data Engine (3단계)

단계	방식	수집량
1단계: Assisted-Manual	전문 어노테이터가 SAM 보조로 수동 라벨링	4.3M 마스크 / 120k 이미지
2단계: Semi-Automatic	자동 마스크 제안 + 어노테이터가 나머지 보완	+5.9M 마스크 / 180k 이미지
3단계: Fully Automatic	32×32 격자 점 프롬프트로 완전 자동화	1.1B 마스크 / 11M 이미지

5. 실험 구성과 결과 (Experiments)

5-1. 23개 데이터셋 제로샷 단일 점 마스크 평가

SAM은 RITM 대비 23개 데이터셋 중 **16개에서 더 높은 mIoU**를 기록했으며, 오라클 평가 (3개 마스크 중 최적 선택)에서는 전 데이터셋에서 RITM을 능가했다.

5-2. 제로샷 엣지 검출 (BSDS500)

SAM은 엣지 검출을 위해 학습되지 않았음에도 HED(BSDS500으로 학습된 딥러닝 방법)에 준하는 성능을 보였다(ODS 0.768 vs HED 0.788).

5-3. 제로샷 오브젝트 프로포절 (LVIS v1)

SAM은 중간/대형 오브젝트 및 희귀/일반 카테고리에서 ViTDet-H(LVIS로 학습된 모델)를 능가하는 성능을 보였다.

5-4. 제로샷 인스턴스 세그멘테이션

ViTDet의 바운딩 박스를 프롬프트로 사용했을 때, 자동 평가 지표(AP)에서는 ViTDet 대비 낮은 성능을 보였으나, 인간 평가자의 마스크 품질 평점에서는 SAM이 ViTDet를 일관되게 능가했다.

5-5. 제로샷 텍스트-투-마스크

CLIP 이미지 임베딩으로 학습하고 추론 시에는 텍스트 임베딩을 사용하는 방식으로, "a wheel", "beaver tooth grille" 같은 단순하거나 전문적인 텍스트 프롬프트로도 마스크를 예측할 수 있었다.

6. 발견사항과 한계점 (Discussion)

발견사항

SAM은 훈련 중 보지 못한 새로운 도메인(수중 이미지, 자아중심 이미지 등)에 대해서도 제로샷으로 강력한 성능을 발휘하며, 프롬프트와 컴포지션을 통해 설계 시 상상하지 못했던 새로운 능력을 발휘할 수 있다.

한계점

SAM은 미세한 구조를 놓치거나 작은 분리된 구성 요소를 잘못 생성하기도 하며, 경계선을 고해상도로 처리하는 전용 메서드보다 덜 선명한 경계를 생성한다. 또한 텍스트-투-마스크는 아직 완전히 robust하지 않으며, 의미론적/파놉틱 세그멘테이션을 위한 단순한 프롬프트 설계 방법이 불분명하다.

7. 결론 및 주요 요약 (Conclusion)

Segment Anything 프로젝트는 이미지 세그멘테이션을 파운데이션 모델의 시대로 끌어올리려는 시도로, 새로운 태스크(Promptable Segmentation), 모델(SAM), 데이터셋(SA-1B)의 세 가지 기여를 통해 이를 실현했다.

기여	내용
SA-1B 데이터셋 공개	11M 이미지, 1.1B 마스크 (기존 최대 대비 400배)
SAM 모델 공개	Apache 2.0 오픈 라이선스
온라인 데모	https://segment-anything.com

8. 기존 연구 대비 차별점

비교 항목	기존 연구	SAM
목표	특정 태스크 (인스턴스/시맨틱 세그)	어떤 세그멘테이션 프롬프트에도 응답
학습 방식	완전 지도학습, 태스크별	프롬프트 시뮬레이션 기반 사전학습
제로샷	불가 혹은 제한적	새로운 도메인·태스크로 제로샷 전이
상호작용 방식	사람 중심 (human-in-the-loop)	알고리즘과도 컴포지션 가능
데이터	기존 공개 데이터셋 활용	자체 데이터 엔진으로 1B+ 마스크 생성
실시간성	대부분 느림	프롬프트 처리 ~50ms (웹 브라우저)

9. 핵심 아이디어

|"Segment Anything = NLP 프롬프팅 패러다임 × 이미지 세그멘테이션"

또 다른 핵심 아이디어는 **이미지 임베딩과 프롬프트 처리의 분리**이다. 무거운 이미지 인코더는 이미지당 한 번만 실행하고, 경량 디코더로 다양한 프롬프트에 실시간 대응하는 구조가 실용성을 담보한다.

10. 주요 수식과 코드 분석

10-1. 손실 함수

마스크 예측은 **Focal Loss**와 **Dice Loss**의 선형 결합으로 지도학습하며, 비율은 20:1(Focal:Dice)이다. IoU 예측 헤드는 MSE 손실로 학습된다.

$$L_{total} = L_{mask} + 1.0 \cdot L_{IoU}$$

모호성 처리 시: 3개 마스크 중 최소 손실만 역전파:

$$L = \min_{k \in \{1, 2, 3\}} L_{mask}(k)$$

10-2. 학습 알고리즘

```
# 학습 시 프롬프트 시뮬레이션 (11라운드)
# 실제 구현 의사코드

def simulate_prompts(gt_mask, num_rounds=11):
    """
    이터레이티브 인터랙티브 세그멘테이션 시뮬레이션
    - 총 11회 반복: 초기 1회 + 8회 이터레이티브 + 2회 마스크 정제
    """

    # 1. 초기 프롬프트: foreground point 또는 bounding box (50% 확률)
    if random() < 0.5:
        # GT 마스크 내부에서 경계로부터 가장 먼 점 선택
        first_prompt = get_center_point(gt_mask) # distance transform 최대값
    else:
        # GT 박스에 노이즈 추가 (표준편차: 박스 변 길이의 10%, 최대 20px)
        first_prompt = get_noisy_box(gt_mask, noise_std=0.1)

    # 2. 이터레이티브 포인트 샘플링 (8회)
    prev_mask_logits = None
    for i in range(8):
        # 에러 영역에서 새로운 점 샘플링
        error_region = compute_error(prev_prediction, gt_mask)

        # False Negative → foreground point
        # False Positive → background point
        new_point = sample_from_error_region(error_region)

        # 이전 마스크 로짓을 추가 프롬프트로 제공 (이진화 X, 연속값 그대로)
        prediction = sam(image, prompts + [new_point], prev_mask_logits)
        prev_mask_logits = prediction.logits
```

```

# 3. 2회 마스크 정제 (새 점 없이 마스크만으로 refine)
for _ in range(2):
    prediction = sam(image, prompts, prev_mask_logits)
    prev_mask_logits = prediction.logits

return prediction

```

10-3. 프롬프트 인코딩

```

# 점 프롬프트 인코딩 (개념적 구현)
def encode_point_prompt(point_coords, is_foreground: bool):
    """
    점 = 위치의 positional encoding + foreground/background
    구분 learned embedding
    """
    # Fourier feature 기반 positional encoding
    pos_enc = fourier_positional_encoding(point_coords) #
    shape: [256]

    # foreground(0) / background(1) 구분 임베딩
    type_embedding = learned_embeddings[0 if is_foreground
    else 1] # shape: [256]

    return pos_enc + type_embedding # element-wise sum

# 박스 프롬프트 인코딩
def encode_box_prompt(box_coords):
    """
    박스 = (top-left 위치 인코딩 + TL 임베딩, bottom-right 위치
    인코딩 + BR 임베딩)
    """
    tl_enc = fourier_positional_encoding(box_coords[:2]) +
    tl_embedding
    br_enc = fourier_positional_encoding(box_coords[2:]) +
    br_embedding
    return [tl_enc, br_enc]

```

10-4. 마스크 디코더 핵심 연산

```
# Mask Decoder 핵심 연산 (각 레이어)
def decoder_layer(tokens, image_embedding):
    """
    tokens: prompt embeddings + output tokens (Ntokens × 25
6)
    image_embedding: 64×64×256 → (4096 × 256)로 flatten
    """

    # Step 1: 토큰 간 self-attention
    tokens = self_attention(tokens, tokens, tokens)

    # Step 2: 토큰→이미지 cross-attention (token이 query)
    # 위치 인코딩을 image_embedding에 더한 후 사용
    tokens = cross_attention(
        query=tokens,
        key=image_embedding + pos_enc,    # 위치 정보 주입
        value=image_embedding
    )

    # Step 3: MLP로 각 토큰 업데이트
    tokens = mlp(tokens)

    # Step 4: 이미지→토큰 cross-attention (image가 query)
    # 프롬프트 정보를 이미지 임베딩에 주입
    image_embedding = cross_attention(
        query=image_embedding + pos_enc,
        key=tokens, # 전체 원본 prompt token 재추가
        value=tokens
    )

    return tokens, image_embedding

# 최종 마스크 예측
def predict_mask(output_token, upscaled_image_embedding):
    """
    dynamic linear classifier: output token → mask
```

```

"""
# MLP로 output token → upscaled embedding과 동일 채널의
벡터
mask_weights = mlp(output_token) # shape: [C]

# point-wise dot product → 각 위치별 마스크 확률
mask = dot_product(upscaled_image_embedding, mask_weights)
return sigmoid(mask)

```

10-5. 자동 마스크 생성 파이프라인

```

def automatic_mask_generation(image, sam_model):
    """
    SA-1B 데이터 생성 및 제로샷 세그멘테이션에 사용
    """

    # 1. 격자 점 프롬프트 생성 (32×32 = 1024개)
    grid_points = generate_grid(32, 32, image.shape)

    # 2. 각 점에서 3개 마스크 예측 (전체/부분/세부부분)
    all_masks = []
    for point in grid_points:
        masks = sam_model.predict(image, point_prompt=point)
    # 3개 반환
    all_masks.extend(masks)

    # 3. NMS로 중복 제거 (IoU threshold=0.7)
    masks = nms(all_masks, threshold=0.7)

    # 4. 품질 필터링
    masks = [m for m in masks if m.predicted_iou >= 0.88]
    # confident masks

    # 5. 안정성 필터링
    masks = [m for m in masks
              if stability_score(m, delta=0.5) >= 0.95] # stable masks

```



```
# 6. 전체 이미지 커버 마스크 제거
masks = [m for m in masks if m.area / image.area < 0.9
5]

# 7. 후처리: 소형 연결 요소 제거 (< 100px), 소형 구멍 채우기
masks = postprocess(masks, min_component_area=100)

return masks
```